

Dynamic Android Malware Detection

Runtime Behavioral Analysis using Frida and Machine Learning

Akhil Kumar Valluru (24BCS018)

Mokshith (24BCS106)

Ranjith Reddy (24BCS057)

Chatur (24BCS021)

Harini (24BCS009)

Department of Computer Science and Engineering
Indian Institute of Information Technology, Design and Manufacturing, Jabalpur

July 2026

Agenda

- 1 Introduction & Motivation
- 2 Problem Statement
- 3 Objectives
- 4 Literature Review
- 5 Proposed Methodology
- 6 Experimental Setup
- 7 References

Introduction & Motivation

Why Android Malware?

- Android is the world's most widely used mobile operating system, making it a major target for cyber attacks [1].
- The increasing popularity of Android has led to a rapid rise in Android malware.
- Modern malware uses techniques such as code obfuscation and dynamic code loading to evade detection.
- Dynamic analysis observes application behaviour during execution, revealing malicious runtime activities.



FRIDA

Motivation

Develop an automated framework that captures runtime behaviour using **Frida** and applies **Machine Learning** to classify Android applications as **Benign** or **Malware** [2].

Problem Statement

Existing Malware Detection Approaches

Static Analysis

- Code Obfuscation
- Dynamic Code Loading
- Packed APKs
- Hidden Malicious Behaviour

Dynamic Analysis

- Runtime Behaviour Monitoring
- Detects Hidden Activities
- Observes Real Execution
- Behaviour-based Detection

Problem

Traditional static analysis techniques are often ineffective against modern Android malware because malicious behaviour frequently appears only during application execution.

Project Goal

Develop a runtime malware detection framework using **Frida** to monitor application behaviour and apply **Machine Learning** to accurately classify the Android applications.

Objectives

- Perform runtime behavioural analysis of Android applications.
- Monitor Android API calls using Frida.
- Generate behavioural JSON logs automatically.
- Extract runtime behavioural features.
- Train Machine Learning models.
- Classify Android applications as Benign or Malware.

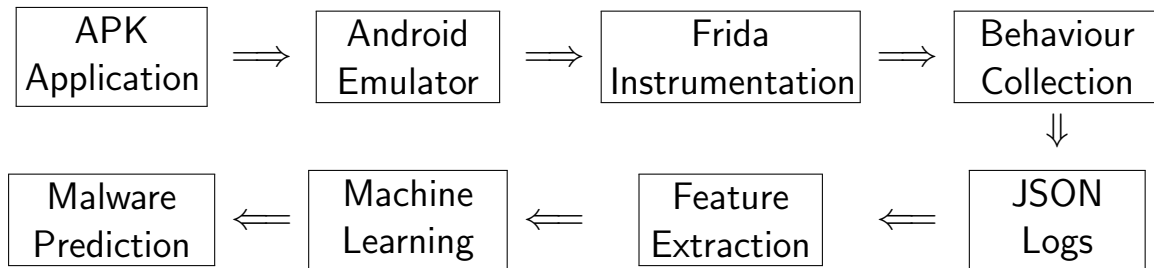
Literature Review

Technique / Tool	Purpose	Limitation
Static Analysis	Examines APK without executing it	Cannot reliably detect dynamically loaded or obfuscated malware
Dynamic Analysis	Observes application behaviour during execution	Requires a controlled execution environment
Frida	Runtime instrumentation and API hooking	Captures runtime behaviour but does not perform classification
Machine Learning	Learns behavioural patterns to classify applications	Performance depends on feature quality and training data

Research Gap

Most existing approaches focus either on static analysis or runtime monitoring. Our work combines **dynamic behavioural analysis using Frida** with **Machine Learning** to automatically classify Android applications as benign or malicious.

Proposed Methodology



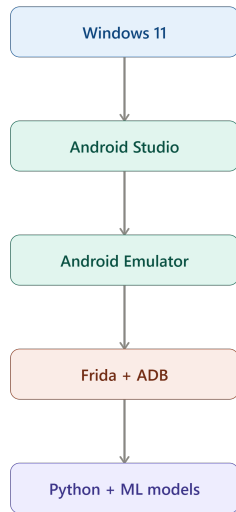
Workflow Summary

The proposed framework executes Android applications inside an emulator, captures runtime API behaviour using Frida, converts the collected behaviour into structured JSON logs, extracts behavioural features, and finally applies Machine Learning models to classify applications as **Benign** or **Malicious**.

Experimental Setup

Software Environment

Component	Specification
Python	Python 3.14.0
Android Studio	Latest Version
Android Emulator	Android Virtual Device (AVD)
ADB	Android Debug Bridge
Frida	Runtime Instrumentation Toolkit
ML Libraries	Scikit-learn, XGBoost, Light-GBM, CatBoost



Experimental Setup (Execution Environment)

Framework Execution Steps

- 1 Launch Android Emulator.
- 2 Start **ADB** connection.
- 3 Execute **Frida Server**.
- 4 Hook Android APIs using Frida.
- 5 Collect runtime behavioural logs.
- 6 Generate JSON output.
- 7 Perform feature extraction for Machine Learning.

Android Emulator executing the target application

VS Code terminal showing Frida instrumentation

Runtime Behaviour Collection

Purpose of Runtime Behaviour Collection

- Capture Android API calls using Frida instrumentation.
- Detect hidden malicious activities that are not visible during static analysis.
- Generate structured JSON logs for feature extraction.
- Prepare high-quality behavioural features for Machine Learning.

Outcome

Runtime monitoring produces behavioural data that forms the input for the feature extraction and malware classification stages.

```
(.venv) ...> frida-ps -U
PID  Name
----  -----
5501  Chrome
1670  Google
...
5878  adbd
...
328   android.hardware.wifi@1.0-service

(.venv) ...> python runner.py

📁 Processing category: benign

📦 Processing:
net.sourceforge.opencamera_96.apk
Performing Streamed Install
Success
Events injected: 100
✅ Raw events captured: 214
📁 Events captured: 214
Success
✅ Saved:
benign_net.sourceforge.opencamera_96.json

🚀 DONE
```

Runtime behaviour collection using Frida instrumentation

Behavioural Data Collection

Behaviour Captured During Execution

- Android API Calls
- File System Operations
- Network Connections
- Dynamic Library Loading
- Permission Usage

Output

All runtime events are collected using Frida and stored in structured JSON format. These behavioural logs serve as the input for the feature extraction stage.

```
{
  "ts": 1784117945077,
  "type": "sp",
  "data": {
    "key": "save_location_history_0"
  }
},
{
  "ts": 1784117945081,
  "type": "sp",
  "data": {
    "key": "preference_save_location"
  }
},
```

Example of behavioural JSON generated by the framework

Feature Extraction

Feature Extraction Process

- ① Read behavioural JSON logs.
- ② Parse runtime API events.
- ③ Compute behavioural statistics.
- ④ Convert behaviours into numerical features.
- ⑤ Generate feature vectors.
- ⑥ Store the dataset in CSV format.

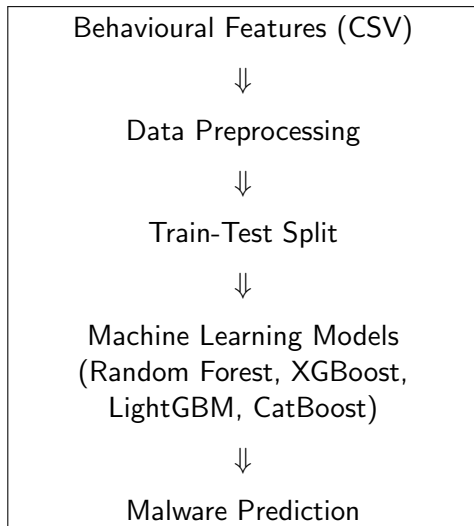
```
file_name,label,num_events,duration,avg_time_between_events,ts_std,ts_entropy,count_activity
malware_com.microsoft.bing.json,0,19,8451.0,469.5,1836.93632996149/2,3.286629485199231,1,0,0
malware_com.mobirix.slideking.json,0,119,10585.0,89.70338983050847,1742.8639199482514,6.5964
malware_com.miniloft.worldcup.json,0,124,17328.0,140.8780487804878,1709.2755325543235,6.1928
malware_com.jacklee.wadual.json,0,58,550.0,9.649122807017545,158.739544114365,5.206697583853
malware_com.jawanikozayanaKijiye.rizo.json,0,68,16262.0,242.71641791044777,1846.347425759573
malware_com.citybeetles.newsottawa.json,0,74,23834.0,326.4931506849315,4415.635463650181,5.8
malware_com.classdojo.android.json,0,63,11469.0,184.98387096774192,3248.633917984817,5.78680
malware_com.citybeetles.newshaiti.json,0,81,21120.0,264.0,4205.3102569201465,5.8075115309505
benign_com.uefa.euro2016.json,0,84,3049.0,36.734939759036145,540.8590466671436,5.41159669628
benign_com.aandrill.president.json,0,286,23770.0,83.40350877192982,10005.765342657565,6.9561
benign_com.whaff.whaffapp.json,0,153,9190.0,60.46052631578947,3792.5086508879162,6.511381517
benign_com.aadhk.time.json,0,52,2019.0,39.588235294117645,571.7794812232578,5.30881417378169
```

Behavioural feature dataset generated from JSON logs

Output

The extracted behavioural features form the input dataset used for Machine Learning based malware classification.

Machine Learning Pipeline



Stage	Description
Input	Behavioural Feature Dataset
Scaling	RobustScaler
Validation	5-Fold Stratified Cross Validation
Models	RF,XGBoost,LightGBM, CatBoost
Output	Benign / Malware Prediction

Observation

The extracted behavioural features are preprocessed and evaluated using multiple Machine Learning algorithms to identify malicious Android applications.

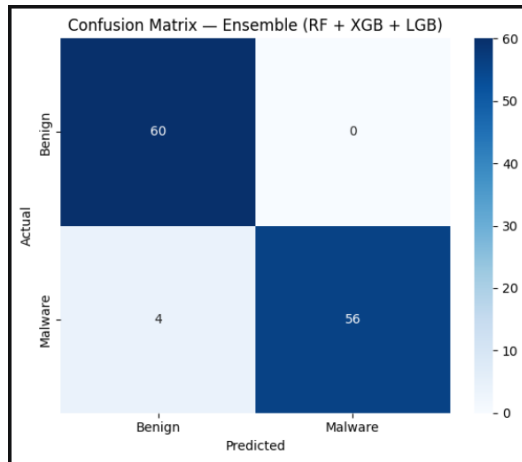
Experimental Results

Performance Evaluation

Metric	Value
Accuracy	0.9667
Precision	1.0000
Recall	0.9333
F1-Score	0.9655
ROC-AUC	0.9844

Key Observations

- High malware detection accuracy.
- Balanced performance across both classes.
- Behavioural features improve classification reliability.



Confusion Matrix of the Best Performing Model

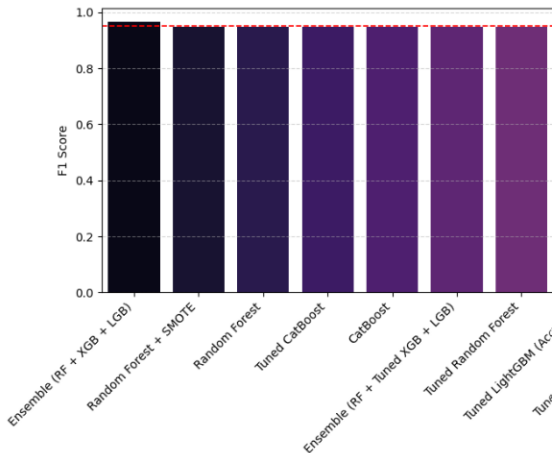
Comparison of Machine Learning Models

Model Performance

Model	Accuracy
Random Forest	95.00%
XGBoost	91.67%
LightGBM	93.33%
CatBoost	95.00%

Observation

The ensemble model combining Random Forest, XGBoost, and LightGBM achieved the highest classification accuracy, outperforming the individual machine learning models.



Accuracy comparison of evaluated models

Challenges Faced

Technical Challenges

- Configuring Frida on the Android emulator for runtime instrumentation.
- Handling emulator crashes and ADB connectivity issues.
- Managing noisy and redundant behavioural events generated during execution.
- Engineering meaningful behavioural features from raw JSON logs.
- Selecting an optimal Machine Learning model among multiple alternatives.

Future Work

- Develop a real-time malware analysis platform capable of detecting and classifying malware from multiple input formats such as APK files, images (PNG, JPG, JPEG), documents, and other digital file formats using advanced AI-driven analysis techniques.
- Evaluate larger and more diverse malware datasets.
- Improve model explainability using Explainable AI (XAI).

Future Direction

The proposed framework can be extended into a lightweight real-time Android security solution capable of detecting emerging malware families.

Conclusion

- Developed a dynamic Android malware detection framework using Frida instrumentation.
- Collected runtime behavioural data from Android applications.
- Generated behavioural feature datasets for Machine Learning.
- Evaluated multiple Machine Learning algorithms and ensemble techniques.
- The ensemble model combining Random Forest, XGBoost, and LightGBM achieved the best detection performance on the balanced Dataset E.

Takeaway

Dynamic behavioural analysis combined with ensemble learning provides an effective approach for Android malware detection.

References

- ① M. V. S. Sharma, M. Tilak, U. Pareek et al., "Explainable AI-Based Android Malware Detection," SN Computer Science, vol. 6, Article 1006, 2025 <https://link.springer.com/article/10.1007/s42979-025-04548-3>
- ② Google. Android Security Bulletin. 2024.
- ③ Ole Andre Vadla Ravnas. Frida Documentation. <https://frida.re>
- ④ Arp, D. et al. DREBIN: Effective and Explainable Detection of Android Malware. NDSS, 2014.
- ⑤ Enck, W. et al. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. ACM TOCS, 2014.
- ⑥ Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.
- ⑦ Chen, T. and Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD, 2016.
- ⑧ Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS, 2017.
- ⑨ Prokhorenkova, L. et al. CatBoost: Unbiased Boosting with Categorical Features. NeurIPS, 2018.
- ⑩ Chawla, N. et al. SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 2002.

Thank You

Questions?

Dynamic Android Malware Detection

IIITDM Jabalpur